CC1101Radio User Manual

Arduino SPI Edition

Updated for the Arduino-only CC1101Radio API

This revision documents the migration away from direct ESP-IDF SPI, GPIO, timer, and FreeRTOS dependencies. The driver is intended to be used through the Arduino framework with Arduino.h, SPI.h, SPIClass, and SPISettings.

The RF configuration and packet API remain centered on the TI CC1101: frequency, modulation, bitrate, deviation, receive bandwidth, packet modes, address filtering, CRC, whitening, Manchester encoding, FEC, output power, RSSI/LQI, and raw register access.

Important API change

```
CC1101Radio radio(SPI, PIN_CS, PIN_MISO);

void setup() {
  SPI.begin(PIN_SCK, PIN_MISO, PIN_MOSI, PIN_CS);
  int16_t rc = radio.begin();
}
```

The sketch owns SPI bus initialization. The radio object receives the SPIClass instance, its CS pin, and optionally the MISO pin used for the CC1101 chip-ready check.

1. What changed

Area	Previous ESP-IDF implementation	Arduino implementation
SPI object	spi_host_device_t / device handle	SPIClass + SPISettings
Transactions	spi_device_acquire_bus / polling transmit	beginTransaction / transfer / endTransaction
GPIO	gpio_config / gpio_set_level / gpio_get_level	pinMode / digitalWrite / digitalRead
Timing	esp_timer / esp_rom delay	micros / millis / delayMicroseconds / delay
RTOS dependency	FreeRTOS types used by bus lock	No direct FreeRTOS dependency
Chip select	Manual GPIO CS	Manual Arduino GPIO CS retained

The key architectural point is that the Arduino sketch initializes the bus. The library controls only its own CS line and brackets CC1101 traffic with Arduino SPI transactions.

2. Installation

Install the library folder with this structure:

```
CC1101Radio/
  library.properties
  src/
    CC1101Radio.h
    CC1101Radio.cpp
  examples/
    BasicReceive/
      BasicReceive.ino
```

In Arduino IDE, either install the packaged ZIP as a library or place the library folder in your Arduino libraries directory. Include it with:

```
#include <Arduino.h>
#include <SPI.h>
#include <CC1101Radio.h>
```

3. Wiring and SPI ownership

The CC1101 remains an SPI peripheral. Connect SCK, MOSI/SI, MISO/SO, CSn, 3.3 V, and ground according to your board and CC1101 module. The driver uses SPI mode 0 and MSB-first transfers.

Manual CS is intentional

CSn is driven with digitalWrite() rather than hardware-controlled CS. This preserves the CC1101 requirement that the SO/MISO ready state can be observed while CS remains low.

MISO ready check

When a valid MISO pin is supplied, the driver waits for SO/MISO to go low after CS is asserted. Pass -1 to skip that GPIO check and use the fallback timing path.

SPI clock

The constructor accepts an SPI clock request, with the driver design limiting burst access to 6.5 MHz. The default remains 4 MHz.

4. Constructor and startup

```
CC1101Radio(SPIClass& spi,
            int csPin,
            int misoPin = -1,
            uint32_t spiHz = 4000000);
```

For the normal ESP32 Arduino SPI instance:

```
CC1101Radio radio(SPI, PIN_CS, PIN_MISO);
```

Startup order

```
void setup() {
  Serial.begin(115200);

  // The sketch owns the bus.
  SPI.begin(PIN_SCK, PIN_MISO, PIN_MOSI, PIN_CS);

  int16_t rc = radio.begin(
      433.92f,
      4.8f,
      47.6f,
      135.0f,
      CC1101Radio::MOD_GFSK);

  if (rc != CC1101Radio::ERR_NONE) {
    Serial.printf("CC1101 init failed: %d\n", rc);
    return;
  }

  radio.startReceive();
}
```

Do not use the old ESP-IDF host-style constructor. You no longer pass SPI2_HOST or an ESP-IDF device handle.

5. SPI transaction behavior

Each public low-level access enters an Arduino SPI transaction using SPI.beginTransaction(settings) and leaves it with SPI.endTransaction(). Internal *Unlocked helpers are used when a higher-level operation needs several CC1101 accesses inside one transaction.

```
BusLock lock(*_spi, _spiSettings);
// ... unlocked CC1101 accesses ...
// destructor calls endTransaction()
```

The byte transport is now based on SPIClass::transfer(). The CC1101-specific command, register, status, strobe, FIFO burst, and packet logic stays above that transport layer.

6. Core API

Group	Methods
Initialization	begin(), reset()
RF configuration	setFrequency(), setBitrate(), setDeviation(), setRxBandwidth(), setModulation()
Channels	setChannelSpacing(), setChannel()
Packet framing	setSyncWord(), setSyncMode(), setPreambleBytes(), setCrc(), setWhitening()

Group	Methods
Coding	setManchester(), setFec()
Addressing	setNodeAddress(), setAddressFilter(), transmitTo()
Length modes	setVariableLength(), setFixedLength(), setInfiniteLength()
Power	setOutputPowerDbm(), setPaRaw(), setInductorType()
State	idle(), sleep(), wake(), calibrate(), startReceive()
TX/RX	transmit(), transmitTo(), readPacket(), readStream(), available()
Diagnostics	getRSSI(), packetStatus(), carrierSense(), frequencyErrorKHz(), partNumber(), version(), marcState()
Raw access	readReg(), writeReg(), readStatus(), strobe(), readBurst(), writeBurst()

7. Basic receive example

```
constexpr int PIN_SCK  = 18;
constexpr int PIN_MISO = 19;
constexpr int PIN_MOSI = 23;
constexpr int PIN_CS   = 5;

CC1101Radio radio(SPI, PIN_CS, PIN_MISO);

void setup() {
  Serial.begin(115200);
  SPI.begin(PIN_SCK, PIN_MISO, PIN_MOSI, PIN_CS);

  int16_t rc = radio.begin(433.92f, 4.8f, 47.6f, 135.0f,
                           CC1101Radio::MOD_GFSK);
  if (rc != CC1101Radio::ERR_NONE) return;

  Serial.printf("PARTNUM=0x%02X VERSION=0x%02X\n",
                radio.partNumber(), radio.version());
  radio.startReceive();
}

void loop() {
  uint8_t data[CC1101Radio::MAX_PAYLOAD];
  CC1101Radio::PacketInfo info{};
  int16_t len = radio.readPacket(data, sizeof(data), &info);

  if (len > 0) {
    Serial.printf("RX %d bytes, RSSI %.1f dBm, LQI %u, CRC %s\n",
                  len, info.rssi, info.lqi, info.crcOk ? "OK" : "BAD");
  }
}
```

8. Multiple radios and alternate SPI buses

Because the constructor receives a SPIClass reference, separate CC1101Radio objects can be attached to the same Arduino SPI bus with different CS pins. A radio can also be constructed against another SPIClass instance supported by the selected Arduino core.

```
CC1101Radio radioA(SPI, CS_A, MISO_PIN);
CC1101Radio radioB(SPI, CS_B, MISO_PIN);
```

Initialize the SPIClass instance before calling begin() on the radios. Each object owns its CC1101 state and CS pin; the sketch owns bus initialization.

9. Migration checklist

Use this checklist when moving an older ESP-IDF sketch to the Arduino version:

```
[ ] Remove driver/spi_master.h and driver/gpio.h from application code
[ ] Remove direct ESP-IDF spi_bus_initialize / spi_bus_add_device setup
[ ] Include Arduino.h and SPI.h
[ ] Initialize the Arduino SPIClass from setup()
[ ] Construct CC1101Radio with (SPIClass&, CS, MISO, optional clock)
[ ] Keep radio.begin(...) for CC1101 RF setup
[ ] Keep existing modulation, packet, power, and RX/TX calls
[ ] Re-enter RX with startReceive() after configuration or transmit as required
```

10. Error codes

Name	Value	Meaning
ERR_NONE	0	Success
ERR_CHIP_NOT_FOUND	-1	VERSION read returned an invalid/not-present value
ERR_BAD_ARG	-2	Unsupported argument or mode combination
ERR_TIMEOUT	-3	Operation timed out
ERR_OVERFLOW	-4	RX FIFO overflow
ERR_CRC	-5	CRC failed when CRC checking is enabled
ERR_BUF_SMALL	-6	Destination buffer is too small
ERR_LENGTH	-7	Packet length is invalid for the active mode
ERR_TX_UNDERFLOW	-8	TX FIFO underflow
ERR_SPI	-9	SPI/CS attachment could not be established

11. Notes for the SPIder project

This Arduino-facing API keeps the CC1101 transport explicit: the sketch selects and initializes the SPI bus, while each radio instance supplies its own CS pin. That fits a multi-radio layout without exposing ESP-IDF SPI handles throughout the application.

The details above describe the updated Arduino API. RF configuration behavior remains governed by the CC1101 settings exposed by the class and should be matched at both ends of a link.

End of manual